

Assignment 8

Elias Samantzis

Group 65

000715-6631, ADS

gussamanel@student.gu.se

Emrik Dunvald

Group 65

020208-5759, ADS

gusdunvem@student.gu.se

Disclaimer: Grammarly was used to correct and assist in structuring grammar.

1 Introduction

Tic-tac-toe is a simple and classic strategy game where two players face off against each other in a 3x3 grid where the goal is to get three of their own marks in a row. The game serves as simple yet efficient grounds for testing the decision-making of AI models. Traditional AI approaches for the game include algorithms such as Minimax, Monte Carlo Search Tree (MCST), and other machine learning-based techniques. In our previous implementation of the game we implemented an MCST with an opponent that utilized a completely random roll-out policy and this led to our model achieving a 94% win rate. While that is very effective, we aim to enhance our models ability by incorporating some machine learning elements to increase its ability to learn and play the game more efficiently. Our goal is to implement a random forest that will enhance the MCST decision making ability by providing better-informed evaluations on board positions.

Our research question:

- How can a Random Forest classifier be implemented into the Monte Carlo Tree Search algorithm to improve the win rate.

2 Motivation

The MCST relies on random simulations, and this can lead to semi-optimal move selections especially in a game where it is facing more complex opponents. By incorporating a trained random forest model the MCST roll-out policy can become more robust making more strategic and efficient choices, possibly leading to better gameplay. Our goal is to evaluate whether the integration of random forest with MCST results in a higher win rate compared to agents that use a traditional MCST

roll-out policy. If our approach results in a higher win rate it would indicate that decision trees contribute to improved performance, such as was observed in AlphaGo's approach, which combined deep learning with the Monte Carlo strategy (Silver et al., 2016).

Looking past Tic-tac-toe, this project can serve as a foundation for enhancing AI's in strategy games by combining machine learning models with traditional search algorithms in board games, real-time strategy games, and AI decision-making in general. On top of that, dissecting the decision-making process of the random forest can help us understand how certain strategies emerge and what limitations might exist.

3 Methods

3.1 Data:

The dataset contains of 255168 Tic-Tac-Toe games, with each row representing a complete match (Therrien, 2024). The Winner column records the outcome: 'X' for player X, 'O' for player O, and '-' for a draw. Moves are recorded in columns Move 1-X to Move 9-X, using a row column format where 0-0 represents the top-left corner and so on. Unused moves are marked with — as usually all moves are not used in one game.

3.2 Data preprocessing:

The dataset used in this project consists of Tic-tac-toe game states and their respective winners. To prepare the data for usage with the Random Forest machine learning model (RF) a preprocessing pipeline was integrated. The following steps were implemented:

Board state encoding: Each Tic-tac-toe game board consists of a 3x3 grid. The function `convert_board_to_array` maps 'X' to 1, 'O' to -1, and empty spaces to 0, producing a flattened numerical array representation of the board to allow the

RF to be able to read the data at each game state.

Feature extraction: The `extract_partial_states` function extracts different iterations of the game by iterating through the sequences of moves in each game and gradually updating the board state. Each intermediary board state was stored as a snapshot which allowed the model to train on specific and individual game iterations rather than only on completed games.

Label encoding: The winner was encoded numerically as '1' for 'X', as -1 for 'O' and '0' for draws.

Data transformation: Using the function `process_data` the extracted board states, through the function `extract_partial_states` were structured into a feature matrix X and a respective label vector y which were then used to train our RF model.

3.3 Code implementation:

The implementation of the Tic-tac-toe game, Monte Carlo Tree Search (MCTS) algorithms consists of the following components:

Game logic: The `TicTacToe` class includes methods that handle the game state, move-making, and victory conditions that follow a straightforward implementation. The game board is represented as a list of lists and helper methods are implemented to determine available moves, winners, and whether the game has ended or not.

MCTS implementation: The MCTS algorithm is implemented through the `MCTSNode` and `MCTS` classes. In these classes the methods for selection, expansion, simulation, and backpropagation are implemented which act as the main logic behind the MCTS algorithm. The function `best_child` is responsible for choosing the most optimal move based on an exploration weight that influences how much the algorithms explore or utilize known victory moves. The Upper confidence Trees (UCT) is the main component behind the selection of the node that the algorithm chooses to take.

Random opponent simulation: An opponent making completely random moves was implemented to allow for benchmarking the performance of various model implementations against the random opponent.

Evaluation methods The methods `evaluate_mcts_vs_random` and `evaluate_mctsrfs_vs_mcts` simulate multiple games where the win, lose, and draw rates of the MCTS were tracked against

either a random opponent, a normal MCTS player or an MCTS with RF player. The number of games and iterations are given as parameters to allow for optimization and evaluation of the influence of the number games and iterations on the performance of the models. The method `tune_ratio` allows for the tuning of multiple parameters in one go to allow for a faster and clearer evaluation of the RF models impact on the MCTS performance. The results of the `tune_ratio` could then be plotted through the `plot_bar` to allow for visualization of these results.

3.4 Random Forest Integration:

In order to improve the decision-making of the standard MCTS implementation a Random Forest classifier was trained on the preprocessed data and then integrated into the MCTS UCT algorithms. The model is in this case trained on all data and not a standard train test split which in most cases would lead to overfitting. We did not think this would be an issue however since the dataset represents all possible games and we want the model to learn as much as possible of the entire game. The implementation includes the following steps:

Model training: The `RandomForestClassifier` was imported and implemented from the Sklearn library and was then trained on the extracted board game states X as inputs and the respective game outcomes y as labels. The trained model was then saved using `joblib` to avoid unnecessary time spent retraining the model after every shutdown of the program.

Parameter tuning: The parameter ratio is used to determine the weight of influence that the Random Forest has on the MCTS decision-making during gaming sessions. On top of that whether the MCTS with Random Forest is playing as a player one or two is tracked with the player variable to allow the model to adapt its strategy accordingly. When playing as player one the model will favor boards which wins the game but when playing as player two, it will favor boards that lose the game since those are the boards where player two win in the data.

Integration into MCTS algorithm: The model was integrated into the `MCTSNode` class via the `compute_random_forrest_prob` method. This method takes the current board state and then predicts the probability of winning from that point. Whether the model is playing as player one or two

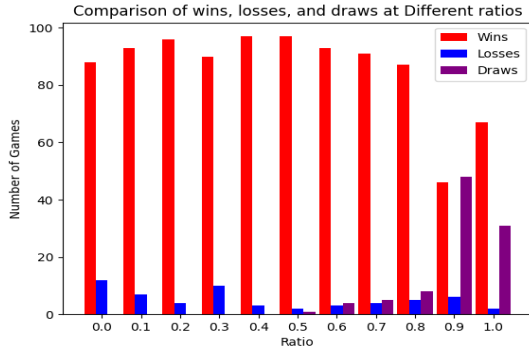


Figure 1: Win rate at different ratios as player 1

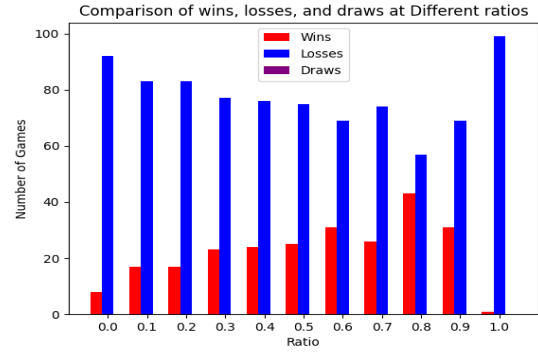


Figure 2: Win rate at different ratios as player 2

is also tracked here with the if statement. This method is then used in the best_child method that we discussed before where the UCT formula is used to select the best node. However, the balance between how much influence the UCT or compute_random_forrest_prob has is determined by the ratio, the higher the ratio, the higher influence the RF model has on the decision-making of the MCTS.

4 Results and Analysis

4.1 Win rate at different ratios as player 1

This data was collected by allowing the new model, which implements a combination of the MCTS strategy and a board evaluation model, to play against the old model. The model was tested at different ratios where the ratio represents the voting power of the random forest classifier. This was done over 100 games at each ratio. When exploring the win rate with different levels of influence from the random forest classifier there were very little improvement as no ratio seemed to increase the win rate. We do however see a decline in win rate at higher ratios of RF influence, suggesting that the model performs worse than the MCTS model. At more even contributions of both models we seem to have the same performance though, suggesting that the random forest classifier is not significantly contributing to the win rate. We suspect that this might be due to already having a significant win rate with the baseline model, around 96%, which is hard to improve. This is however unclear and would require more a more in depth evaluation in order to determine the root cause of the phenomena.

4.2 Win rate at different ratios as player 2

The same evaluation was done as player two where we compared the performance of the model, in this case the number of won games, to the MCTS model. Similarly here, the model had to play as player two which we found to be a significant disadvantage that our previous model struggled with. We once again tested this for 100 games at each ratio. Here we however see a more significant change in the number of won games as the model seems to improve the overall performance. Looking at this data it would appear that a ratio of 0.8 or 80% yields the best result for our new model. The reason for this improvement seems to be that the new model has more information about the games and is able to block player one from winning. Looking at a few example games we observed that our previous model was very greedy when playing as player two, missing opportunities to block the opponent from winning which likely is why we are seeing this increase in performance from the new model.

Compared to the previous scenario we have more room for improvement here which might also be a reason as for why we are seeing an increase in number of wins. Finally, it is possible that the random forest model is better at predicting wins for player two due to having fewer games that lead to this scenario in the data. Since most games result in a win for player one, it could be the case that board evaluation is less useful in this scenario since most boards lead to a win for player one. When instead looking at games where player two wins, we have a smaller number of options which the random forest model learns has more importance.

4.3 Random Forrest Disadvantages

When testing the various models we noted a few disadvantages to utilizing the newer more complex model. Primarily we observed that the model was significantly slower than the baseline model. Depending on the ratio the combined model would be somewhere between one and two seconds to play one game while the old model would do the same in 0.2 seconds. Not significant when we would play one or two games against the model but testing thousands of games did make this a bit of an issue. It might also be a limitation if we were to expand the model to more complex games.

The second thing we observed was that allowing the random forest model have full control of the decision making led to significantly worse performance in terms of win rate. As seen in figure 1 and figure 2, the model performed worse in both scenarios. We believe that this is due to the fact that the random forest model lacks the ability to plan ahead more than one move. The model has full knowledge of every board position but does not value long term strategies which lead to victory. It only picks boards which are likely to win which does not appear to be enough to perform consistently.

4.4 Statistical Significance

In order to verify whether our model was significantly better in terms of number of won games, we computed a one-tailed binomial test. This was done on a sample of 200 games where both the new and old model were allowed to play as player one and two. The ratios were set differently between the two tests since we previously observed that the model appeared to favour different ratios depending on the player role. When playing as player one the ratio was set to 0.5 but when playing as player two it was set to 0.8. The results of these games can be seen in table 1. We use an alpha of 0.05 as the significance level. The null hypothesis and alternative hypothesis apply to both the scenario but we will have two different results since we will conduct the test for both scenario, playing as both player one or two.

- H_0 :The new model does not have a higher probability of winning a game
- H_A :The new model has a higher probability of winning a game

Model	player	Win Rate
MCTS	1	0.960
MCTS+RF	1	0.965
MCTS	2	0.070
MCTS+RF	2	0.450

Table 1: Win Rates of different model in different positions

Player	P Value	α
1	0.423	0.05
2	$1.43 * e^{-49}$	0.05

Table 2: Results of binomial test as player 1 and 2

We see that the model, when playing as player one, has a win rate of 95.6% which is slightly better than the base model but since the P-value in this case is larger than 0.05, we fail to reject the null hypothesis. We therefore assume that the model is not statistically better when playing as player one which is to be expected. However, when playing as player two we have a win rate of 45% which the binomial test shows is statistically significant since $p < \alpha$. We can therefore reject the null hypothesis and say that our model is more likely to win when playing as player two when compared to our previous model with a confidence level of $\alpha = 0.05$.

5 Limitations

While the approach demonstrates the effectiveness of integrating machine learning with classical search-based methods, we encountered several limitations.

Our MCTS with Random forest implementation suffers greatly when the decision-making is 100% on the Random forest. Without the properties of the MCTS, the Random forest is not capable of making the necessary decision in order to win.

One limitation we discovered is the inherent limitations Tic-Tac-Toe has as a game. Player X who moves first, has a significant advantage, which might lead to skewed results in the model evaluation. What might first seem like a very stable model with a high win rate might be completely overturned if the order is switched around. Since optimal play from both sides would result in a draw, the model may overfit to specific move sequences rather than generalize well across different strategies.

The model also suffers from a lack of interpretability as the random forest implementation in MCTS does not have any maximum search depth, meaning that it will continue expanding nodes indefinitely until it reaches its goal. While this allows for a deeper exploration of possibilities it reduces the interpretability and control over the computation time. To try and visualize the decision-making of the ensemble would require extremely large computations, time, and power if it is even possible in that case. On top of that, the Random Forest model is trained on a large dataset which may introduce even more complexity when trying to analyze its decision making pathways. What we could do is visualize the feature importance to see what kinds of moves it favors although it would not give great insight into the overall decision making of the combined model.

Finally, the number of iterations for the MCTS was set to 500 in order to balance the computational power required and performance. While increasing the iteration count could perhaps improve the decision-making even more, the time taken to simulate would grow significantly. The chosen iteration count was deemed enough for our purpose of the algorithm.

6 Future work

One potential direction would be to evaluate the performance of the Random Forest MCTS against a deep machine-learned-based approach, such as the neural network used in the AlphaGo (Silver et al., 2016). Or maybe another simple deep-learned model trained on the same dataset. This could provide insight into whether a more complex model could further improve the decision-making and generalizability of our model.

Another exploration direction would be to explore why the Random Forest attains such poor results when relying on 100% on the decision-making of the Random Forest while playing Tic-Tac-Toe.

Lastly, future work could explore ways to improve and refine the integration of the Random Forest into the MCTS by looking into the weighting of the influence of the Random Forest to find a way to further optimize the perfect parameters. Additional search constraints, such as `max_depth` could be explored to see if the win rate improves after a certain depth or if the computational cost stays the same.

7 Conclusion

In conclusion we can evaluate the results of this paper and see that we partially were able to meet our goal of improving the Monte Carlo model. The original question was how the model could be improved by integrating a random forest classifier for board evaluation and these results do answer this. Implementing the model by first training it on a large number of games and then tuning the search strategy to include the estimated probability of winning with a specific board can improve the win rate of the overall model. The results do however show that this only is the case for the model when playing as player two, showing no statistically significant improvement when playing as player one. It is also worth noting that this only was the case after tuning the models decision making as the influence of the random forest classifier could in some cases lower the probability of winning. This suggests that both models work complementary to one another and that there is no individual model that has a higher probability of winning in all scenarios. In this regard there could certainly be possible to further improve the model and tune more parameters such as exploration weight, MCTS iterations and various parameters of the random forest model as we do not fully understand what factors contribute to the combined models success.

References

- David Silver, Aja Huang, Christopher J. Maddison, Arthur Guez, Laurent Sifre, George van den Driessche, Julian Schrittwieser, Ioannis Antonoglou, Veda Panneershelvam, Marc Lanctot, Sander Dieleman, Dominik Grewe, John Nham, Nal Kalchbrenner, Ilya Sutskever, Timothy Lillicrap, Madeleine Leach, Koray Kavukcuoglu, Thore Graepel, and Demis Hassabis. 2016. [http://www.nature.com/nature/journal/v529/n7587/full/nature16961.h](http://www.nature.com/nature/journal/v529/n7587/full/nature16961.html) Mastering the game of go with deep neural networks and tree search. *Nature*, 529:484–503.
- Anthony Therrien. 2024. <https://doi.org/10.34740/KAGGLE/DSV/10202502> Tic-tac-toe game dataset.

8 Appendix

Listing 1: Code

```

1 import numpy as np
2 import pandas as pd
3 import random
4 import copy
5 import joblib
6 from tqdm import tqdm
7 import matplotlib.pyplot as plt
8 from scipy.stats import binomtest
9 from sklearn.ensemble import RandomForestClassifier
10
11 def convert_board_to_array(board):
12     board_array = np.zeros((3, 3), dtype=int)
13     for row in range(3):
14         for col in range(3):
15             if board[row][col] == 'X':
16                 board_array[row][col] = 1
17             elif board[row][col] == 'O':
18                 board_array[row][col] = -1
19
20     return board_array.flatten()
21
22
23 def extract_partial_states(row):
24     board = np.zeros((3, 3), dtype=int)
25     winner = row["Winner"]
26     partial_states = []
27
28     moves = row[1:].values # skip the winner column
29
30     # go through all moves in the dataset
31     for i, move in enumerate(moves):
32         if move == "---":
33             break
34         row_idx, col_idx = map(int, move.split("-"))
35         player = 1 if i % 2 == 0 else -1 # X = 1, O = -1
36         board[row_idx, col_idx] = player
37
38         # store the board so far in the new dataset
39         partial_states.append((board.flatten().tolist(), winner))
40
41     return partial_states
42
43 def process_data(df):
44     all_data = []
45
46     # go though all rows and extract the board state at each position
47     for _, row in df.iterrows():
48         all_data.extend(extract_partial_states(row))
49
50     # convert to df
51     expanded_df = pd.DataFrame(all_data, columns=["board_state", "winner"])
52     expanded_df["winner"] = expanded_df["winner"].map({"X": 1, "O": -1, "-": 0})
53
54     # flatten board
55     X = np.vstack(expanded_df["board_state"].values)
56     y = expanded_df["winner"].values
57
58     return(X, y)
59
60
61 class TicTacToe:
62     def __init__(self):
63         self.board = [['_'] for _ in range(3)] for _ in range(3)]
64         self.current_player = 'X'
65
66     def available_moves(self):
67         return [(r, c) for r in range(3) for c in range(3) if self.board[r][c] == '_']

```

```

68
69 def make_move(self, row, col):
70     if self.board[row][col] == ' ':
71         self.board[row][col] = self.current_player
72         self.current_player = 'O' if self.current_player == 'X' else 'X'
73         return True
74     return False
75
76 def is_winner(self, player):
77     for row in range(3):
78         if all(self.board[row][col] == player for col in range(3)):
79             return True
80     for col in range(3):
81         if all(self.board[row][col] == player for row in range(3)):
82             return True
83     if all(self.board[i][i] == player for i in range(3)) or \
84         all(self.board[i][2 - i] == player for i in range(3)):
85         return True
86     return False
87
88 def is_draw(self):
89     return all(self.board[row][col] != ' ' for row in range(3) for col in range
90                (3))
91
92 def is_game_over(self):
93     return self.is_winner('X') or self.is_winner('O') or self.is_draw()
94
95 def clone(self):
96     clone = TicTacToe()
97     clone.board = copy.deepcopy(self.board)
98     clone.current_player = self.current_player
99     return clone
100
101 def print_board(self):
102     for row in self.board:
103         print(' '.join(row))
104         print('-' * 5)
105
106 class MCTSNode:
107     def __init__(self, game, parent=None, model=None, ratio=0.0, player=1):
108         self.game = game
109         self.parent = parent
110         self.children = []
111         self.visits = 0
112         self.wins = 0
113         self.model = model
114         self.ratio = ratio
115         self.random_forest_probability = 0
116         self.player = player
117
118     def compute_random_forrest_prob(self):
119         array = [convert_board_to_array(self.game.board)]
120         probabilities = np.clip(self.model.predict_proba(array), 1e-10, 1)
121         if self.player == 1:
122             win_prob = probabilities[0][1] # If the model is player 1, compute the
123             probability of the board winning
124         else:
125             win_prob = probabilities[0][2] # If the model is player 2, compute the
126             probability of the board loosing
127         return win_prob * 10 # mult with 10 so that both MCTS and RF values have
128         similar magnitudes
129
130     def is_fully_expanded(self):
131         return len(self.children) == len(self.game.available_moves())
132
133     def best_child(self, exploration_weight=1.0):
134         max_child = max(self.children, key=lambda child: (child.wins / (child.visits
135             + 1e-6)) +
136             exploration_weight * (2 * (2 * (child.parent.visits + 1) / (
137                 child.visits + 1)) ** 0.5 * (1 - child.ratio) +

```

```

132         child.random_forest_probability * child.ratio)) # add random
           forrest p * w
133     return max_child
134
135     def backpropagate(self, result):
136         # only compute rf probability if necessary
137         if self.visits < 1 and self.model: self.random_forest_probability = self.
           compute_random_forrest_prob()
138         self.visits += 1
139         self.wins += result
140         if self.parent:
141             self.parent.backpropagate(1 - result)
142
143     def expand(self):
144         tried_moves = [child.game.board for child in self.children]
145         for move in self.game.available_moves():
146             new_game = self.game.clone()
147             new_game.make_move(*move)
148             if new_game.board not in tried_moves:
149                 child_node = MCTSNode(new_game, parent=self, model=self.model, ratio
                   =self.ratio, player=self.player)
150                 self.children.append(child_node)
151             return child_node
152         return None
153
154     def simulate(self):
155         simulation_game = self.game.clone()
156         while not simulation_game.is_game_over():
157             moves = simulation_game.available_moves()
158             row, col = random.choice(moves)
159             simulation_game.make_move(row, col)
160             if simulation_game.is_winner('X'):
161                 return 1
162             elif simulation_game.is_winner('O'):
163                 return 0
164             return 0.5 # draw
165
166 class MCTS:
167     def __init__(self, game, iterations=1000, model=None, ratio=0.0,
           exploration_weight=0.0, player=1):
168         self.root = MCTSNode(game, model=model, ratio=ratio, player=player)
169         self.iterations = iterations
170         self.exploration_weight = exploration_weight
171
172     def run(self):
173         for _ in range(self.iterations):
174             node = self.select()
175             if not node.game.is_game_over():
176                 node = node.expand()
177             result = node.simulate()
178             node.backpropagate(result)
179
180     def select(self):
181         node = self.root
182         while node.is_fully_expanded() and node.children:
183             node = node.best_child()
184         return node
185
186     def best_move(self):
187         self.run()
188         best_child = self.root.best_child(exploration_weight=self.exploration_weight
           )
189         return best_child.game
190
191     def random_opponent_move(game):
192         row, col = random.choice(game.available_moves())
193         game.make_move(row, col)
194
195 # Test a single game to see how it looks
196 def play_game(model=None, ratio=0.0):

```



```

197 game = TicTacToe()
198 mcts = MCTS(game, iterations=500, model=model, ratio=ratio) # 500 iterations
    for MCTS
199
200 while not game.is_game_over():
201     game.print_board()
202     if game.current_player == 'X':
203         print("MCTS_Player's_turn(X)")
204         mcts = MCTS(game, iterations=1000, exploration_weight=1.0) # Refresh
            MCTS for the new game state
205         game = mcts.best_move()
206     else:
207         print("Random_Player's_turn(O)")
208         mcts = MCTS(game, iterations=1000, model=model, ratio=ratio,
            exploration_weight=1.0, player=2)
209         game = mcts.best_move()
210
211 game.print_board()
212
213 if game.is_winner('X'):
214     print("MCTS_Player(X)_wins!")
215 elif game.is_winner('O'):
216     print("Random_Player(O)_wins!")
217 else:
218     print("It's_a_draw!")
219
220 # Evaluate vs random, not used
221 def evaluate_mcts_vs_random(num_games=100, mcts_iterations=500, model=None, ratio
    =0.0):
222     mcts_wins = 0
223     random_wins = 0
224     draws = 0
225
226     for game_index in tqdm(range(num_games)):
227         game = TicTacToe()
228         mcts = MCTS(game, iterations=500, model=model, ratio=ratio)
229
230         while not game.is_game_over():
231             if game.current_player == 'X':
232                 # MCTS Player's Turn
233                 mcts = MCTS(game, iterations=500, model=model, ratio=ratio)
234                 game = mcts.best_move()
235             else:
236                 # Random Player's Turn
237                 random_opponent_move(game)
238
239             if game.is_winner('X'):
240                 mcts_wins += 1
241             elif game.is_winner('O'):
242                 random_wins += 1
243             else:
244                 draws += 1
245
246 def evaluate_mctsrfs_vs_mcts(num_games=100, mcts_iterations=500, model=None, ratio
    =0.0):
247     player_1_wins = 0
248     player_2_wins = 0
249     draws = 0
250
251     for _ in tqdm(range(num_games)):
252         game = TicTacToe()
253
254         while not game.is_game_over():
255             if game.current_player == 'X':
256                 # MCTSrfs Player's Turn
257                 mcts = MCTS(game, iterations=500, model=model, ratio=ratio, player
                    =1)
258                 game = mcts.best_move()
259             else:
260                 # MCTS Player's Turn

```

```

261         mcts = MCTS(game, iterations=500)
262         game = mcts.best_move()
263         if game.is_winner('X'):
264             player_1_wins += 1
265         elif game.is_winner('O'):
266             player_2_wins += 1
267         else:
268             draws += 1
269
270     print(f"Results_after_{num_games}_games:")
271     print(f"MCTSrf_Wins:_{player_1_wins}_{(player_1_wins/_num_games:.2%)}")
272     print(f"MCTS_Player_Wins:_{player_2_wins}_{(player_2_wins/_num_games:.2%)}")
273     print(f"Draws:_{draws}_{(draws/_num_games:.2%)}")
274     return player_1_wins, player_2_wins
275
276 def evaluate_mcts_vs_mctsrfs(num_games=100, mcts_iterations=500, model=None, ratio
=0.0):
277     player_1_wins = 0
278     player_2_wins = 0
279     draws = 0
280
281     for _ in tqdm(range(num_games)):
282         game = TicTacToe()
283
284         while not game.is_game_over():
285             if game.current_player == 'X':
286                 # MCTSrf Player's Turn
287                 mcts = MCTS(game, iterations=500)
288                 game = mcts.best_move()
289             else:
290                 # MCTS Player's Turn
291                 mcts = MCTS(game, iterations=500, model=model, ratio=ratio, player
=2)
292                 game = mcts.best_move()
293             if game.is_winner('X'):
294                 player_1_wins += 1
295             elif game.is_winner('O'):
296                 player_2_wins += 1
297             else:
298                 draws += 1
299
300     print(f"Results_after_{num_games}_games:")
301     print(f"MCTS_Wins:_{player_1_wins}_{(player_1_wins/_num_games:.2%)}")
302     print(f"MCTSrf_Player_Wins:_{player_2_wins}_{(player_2_wins/_num_games:.2%)}")
303     print(f"Draws:_{draws}_{(draws/_num_games:.2%)}")
304     return player_1_wins, player_2_wins
305
306 def evaluate_mcts_vs_mcts(num_games=100, mcts_iterations=500):
307     player_1_wins = 0
308     player_2_wins = 0
309     draws = 0
310
311     for _ in tqdm(range(num_games)):
312         game = TicTacToe()
313
314         while not game.is_game_over():
315             if game.current_player == 'X':
316                 # MCTSrf Player's Turn
317                 mcts = MCTS(game, iterations=500)
318                 game = mcts.best_move()
319             else:
320                 # MCTS Player's Turn
321                 mcts = MCTS(game, iterations=500)
322                 game = mcts.best_move()
323             if game.is_winner('X'):
324                 player_1_wins += 1
325             elif game.is_winner('O'):
326                 player_2_wins += 1
327             else:
328                 draws += 1

```

```

329
330     print(f"Results_after_{num_games}_games:")
331     print(f"MCTS_Wins_First:_{player_1_wins}_{(player_1_wins/_num_games:.2%)}")
332     print(f"MCTS_Wins_Second:_{player_2_wins}_{(player_2_wins/_num_games:.2%)}")
333     print(f"Draws:_{draws}_{(draws/_num_games:.2%)}")
334     return player_1_wins, player_2_wins
335
336 def tune_ratio_player2(steps=1, num_games=100, mcts_iterations=500, model=None):
337
338     # wins losses and draws is based on random forrest + mcts playing as player 2
339     wins, losses, total_draws = [], [], []
340
341     for i in range(steps + 1):
342         ratio = i / steps
343         print(f'Testing_random_forrest+_mcts_with_a_ratio_of_{ratio}')
344
345         player_1_wins = 0
346         player_2_wins = 0
347         draws = 0
348         for _ in tqdm(range(num_games)):
349             game = TicTacToe()
350
351             while not game.is_game_over():
352                 if game.current_player == 'X':
353                     # MCTSrf Player's Turn
354                     mcts = MCTS(game, iterations=500)
355                     game = mcts.best_move()
356                 else:
357                     # MCTS Player's Turn
358                     mcts = MCTS(game, iterations=500, model=model, ratio=ratio)
359                     game = mcts.best_move()
360                 if game.is_winner('X'):
361                     player_1_wins += 1
362                 elif game.is_winner('O'):
363                     player_2_wins += 1
364                 else:
365                     draws += 1
366
367             wins.append(player_2_wins)
368             losses.append(player_1_wins)
369             total_draws.append(draws)
370
371     return wins, losses, total_draws
372
373 def tune_ratio_player1(steps=1, num_games=100, mcts_iterations=500, model=None):
374
375     # wins losses and draws is based on random forrest + mcts playing as player 1
376     wins, losses, total_draws = [], [], []
377
378     for i in range(steps + 1):
379         ratio = i / steps
380         print(f'Testing_random_forrest+_mcts_with_a_ratio_of_{ratio}')
381
382         player_1_wins = 0
383         player_2_wins = 0
384         draws = 0
385         for _ in tqdm(range(num_games)):
386             game = TicTacToe()
387
388             while not game.is_game_over():
389                 if game.current_player == 'X':
390                     # MCTSrf Player's Turn
391                     mcts = MCTS(game, iterations=500, model=model, ratio=ratio)
392                     game = mcts.best_move()
393                 else:
394                     # MCTS Player's Turn
395                     mcts = MCTS(game, iterations=500)
396                     game = mcts.best_move()
397                 if game.is_winner('X'):
398                     player_1_wins += 1

```

```

399         elif game.is_winner('O'):
400             player_2_wins += 1
401         else:
402             draws += 1
403
404         wins.append(player_1_wins)
405         losses.append(player_2_wins)
406         total_draws.append(draws)
407
408     return wins, losses, total_draws
409
410 def tune_exploration_player2(steps=1, num_games=100, mcts_iterations=500, model=None
411 , ratio=0.0):
412     wins, losses, total_draws = [], [], []
413
414     exploration_weights = np.linspace(0.1, 2.0, steps)
415
416     for exploration_weight in exploration_weights:
417         print(f'Testing_random_forrest_{mcts}_with_{a}_exploration_weight_of_{
418             exploration_weight}')
419
420         mcts_wins = 0
421         random_wins = 0
422         draws = 0
423         for _ in tqdm(range(num_games)):
424             game = TicTacToe()
425
426             while not game.is_game_over():
427                 if game.current_player == 'X':
428                     # MCTSrf Player's Turn
429                     mcts = MCTS(game, iterations=500)
430                     game = mcts.best_move()
431                 else:
432                     # MCTS Player's Turn
433                     mcts = MCTS(game, iterations=500, model=model, ratio=ratio,
434                                 exploration_weight=exploration_weight, player=2)
435                     game = mcts.best_move()
436                 if game.is_winner('X'):
437                     mcts_wins += 1
438                 elif game.is_winner('O'):
439                     random_wins += 1
440                 else:
441                     draws += 1
442
443             wins.append(random_wins)
444             losses.append(mcts_wins)
445             total_draws.append(draws)
446
447     return wins, losses, total_draws
448
449 def tune_exploration_player1(steps=1, num_games=100, mcts_iterations=500, model=None
450 , ratio=0.0):
451     wins, losses, total_draws = [], [], []
452
453     exploration_weights = np.linspace(0.1, 2.0, steps)
454
455     for exploration_weight in exploration_weights:
456         print(f'Testing_random_forrest_{mcts}_with_{a}_exploration_weight_of_{
457             exploration_weight}')
458
459         mcts_wins = 0
460         random_wins = 0
461         draws = 0
462         for _ in tqdm(range(num_games)):
463             game = TicTacToe()
464
465             while not game.is_game_over():
466                 if game.current_player == 'X':
467                     # MCTSrf Player's Turn
468                     mcts = MCTS(game, iterations=500, model=model, ratio=ratio,

```

```

464         exploration_weight=exploration_weight, player=1)
465         game = mcts.best_move()
466     else:
467         # MCTS Player's Turn
468         mcts = MCTS(game, iterations=500)
469         game = mcts.best_move()
470     if game.is_winner('X'):
471         mcts_wins += 1
472     elif game.is_winner('O'):
473         random_wins += 1
474     else:
475         draws += 1
476
477     wins.append(mcts_wins)
478     losses.append(random_wins)
479     total_draws.append(draws)
480
481     return wins, losses, total_draws
482
483 def plot_ratio(win, loss, draw):
484     x = np.arange(0, 11)
485     width = 0.3
486
487     # Plot bars
488     plt.bar(x - width, win, width=width, label="Wins", color='red')
489     plt.bar(x, loss, width=width, label="Losses", color='blue')
490     plt.bar(x + width, draw, width=width, label="Draws", color='purple')
491
492     # Labels and Title
493     plt.xlabel("Ratio")
494     plt.ylabel("Number_of_Games")
495     plt.title("Comparison_of_wins,_losses,_and_draws_at_Different_ratios")
496     plt.xticks(x, [f"{i/10:.1f}" for i in x])
497     plt.legend()
498     plt.show()
499
500 def plot_exploration(win, loss, draw):
501     x = np.linspace(0.1, 2.0, len(win))
502     width = 0.06
503
504     # Plot bars
505     plt.bar(x - width, win, width=width, label="Wins", color='red')
506     plt.bar(x, loss, width=width, label="Losses", color='blue')
507     plt.bar(x + width, draw, width=width, label="Draws", color='purple')
508
509     # Labels and Title
510     plt.xlabel("Exploration_weight")
511     plt.ylabel("Number_of_Games")
512     plt.title("Comparison_of_wins,_losses,_and_draws_at_Different_exploration_
513               weights")
514     plt.xticks(x, [f"{i:.1f}" for i in x])
515     plt.legend()
516     plt.show()
517
518 def plot_lin(iteration_time):
519     x = np.arange(1, 11)
520
521     plt.plot(x, iteration_time, marker='o')
522
523     # Labels and Title
524     plt.xlabel("Ratio")
525     plt.ylabel("Seconds_per_iteration")
526     plt.title("Comparing_speed_of_the_model_at_different_ratios")
527     plt.xticks(x, [f"{i/10:.1f}" for i in x])
528     plt.grid()
529     plt.show()
530
531 def test_significance(model=None):
532     total_games = 200

```

```

532 # New agents win rate when going first and second
533 wins_first, _ = evaluate_mctsrfs_vs_mcts(num_games=total_games, mcts_iterations
    =500, model=model, ratio=0.5)
534 _, wins_second = evaluate_mcts_vs_mctsrfs(num_games=total_games, mcts_iterations
    =500, model=model, ratio=0.8)
535
536 mcts_wins_first, mcts_wins_second = evaluate_mcts_vs_mcts(num_games=total_games,
    mcts_iterations=500)
537
538 # Baseline win rates of the old agent
539 P_first = mcts_wins_first / total_games
540 P_second = mcts_wins_second / total_games
541
542 # Binomial test
543 p_value_first = binomtest(wins_first, total_games, p=P_first, alternative='
    greater')
544 p_value_second = binomtest(wins_second, total_games, p=P_second, alternative='
    greater')
545
546
547 print(f"P-value_ (First_move_advantage_test):_{p_value_first}")
548 print(f"P-value_ (Second_move_disadvantage_test):_{p_value_second}")
549
550 def main():
551     # Process data
552     games_df = pd.read_csv('tictactoe_games.csv')
553     X, y = process_data(games_df)
554
555     # Train model and save
556     rf_model = RandomForestClassifier(random_state=0)
557     rf_model.fit(X, y)
558     joblib.dump(rf_model, "random_forest_tictactoe.pkl")
559
560     # Load model
561     rf_model = joblib.load("random_forest_tictactoe.pkl")
562
563     # Play 1 game for debugging
564     play_game(model=rf_model, ratio=0.5)
565
566     # Test ratio as player 2
567     wins, losses, draws = tune_ratio_player2(steps=10, num_games=100,
        mcts_iterations=500, model=rf_model)
568     plot_ratio(wins, losses, draws)
569
570     # Test ratio as player 1
571     wins, losses, draws = tune_ratio_player1(steps=10, num_games=100,
        mcts_iterations=500, model=rf_model)
572     plot_ratio(wins, losses, draws)
573
574     # Test exploration, not used
575     wins, losses, draws = tune_exploration_player1(steps=10, num_games=100,
        mcts_iterations=500, model=rf_model, ratio=0.8)
576     plot_exploration(wins, losses, draws)
577
578     # Test statistical significance of model performance
579     test_significance(model=rf_model)
580
581     # Evaluate model, debugging
582     """_, _ = evaluate_mcts_vs_mcts(num_games=100, mcts_iterations=500)
583     _, _ = evaluate_mctsrfs_vs_mcts(num_games=10, mcts_iterations=500, model=rf_model
        , ratio=0.8)
584     _, _ = evaluate_mcts_vs_mctsrfs(num_games=10, mcts_iterations=500, model=rf_model
        , ratio=0.8)"""
585
586
587 if __name__ == "__main__":
588     main()

```